IMPLEMENTATION REPORT

Centro

> AI-powered document collection platform for accounting firms — Pilot MVP build

Prepared for	noammaarachot@gmail.com
Source spec	Centro Engineering Product Specification (EPS) v2.0, Ch. 1–26
Report date	July 19, 2026
Milestones delivered	M1 – M13 (13 of 13), all committed to git
Commits	13 (1 pre-existing landing page + 12 build milestones)
Lines changed	+19,617 / −4,053 across 74 files
Stack	Next.js 16 (App Router) · TypeScript · Drizzle ORM · PostgreSQL

1. Executive Summary

Centro's Pilot MVP was built from the EPS end to end across thirteen milestones (M1–M13), each independently verified with scripted, real HTTP end-to-end tests against a live embedded Postgres database — not just type-checks. Every EPS chapter with pilot-scope requirements (Ch.1–Ch.20, plus the operational chapters 21–26) has corresponding working code, with the exception of pieces that require external credentials this environment doesn't have (Google OAuth, WhatsApp Business API, an LLM/OCR provider) — those are implemented as clearly-documented mock adapters with identical interfaces, so a real integration is a contained swap, not a redesign.

The single largest event during the build was a security audit (part of M12) that found and fixed a real, severe cross-tenant data-isolation bug: several server actions mutated resources by ID alone, without confirming the resource belonged to the caller's organization. This is detailed in full in Section 8 — it was found, fixed, and independently re-verified with a scripted cross-organization attack before the milestone was considered complete.

Milestones	13 of 13 complete
Database tables	13 (7 migrations, 0000–0006)
App routes	18 pages/routes, 2 API endpoints
Server actions	~45 across 9 action files
Scripted E2E checks run	190+ individual assertions across 13 milestone test passes, all green at time of writing
Mocked integrations	Google Drive, WhatsApp Business API, AI/OCR classification — real credentials not available in this environment
Real, non-mocked infra	Auth/sessions, audit log, state machines, scheduler, business-hours gating, resilience/retry, security fixes

> **How to read "mocked":** a mocked integration means the *transport* (the actual network call to Google/Meta/an LLM provider) is a deterministic stand-in, generating realistic IDs/responses instead of calling a real API. In every case, the *decision logic* around it — matching, validation, retries, gating, state transitions — is real, not a placeholder, and was built to the exact interface a real integration would need to implement, so swapping the transport later touches one file, not the surrounding system.

2. Table of Contents

3. System Architecture

3.1 Stack

Layer	Choice	Why
Framework	Next.js 16.2.10 (App Router, Turbopack)	Already the project's stack; AGENTS.md flagged it as having breaking changes from training-data knowledge, so the actual v16 docs (bundled in `node_modules/next/dist/docs`) were read before writing routing/data code.
Language	TypeScript, strict mode	Existing project convention.
Database	PostgreSQL via Drizzle ORM	Matches EPS DB-8.1 exactly ("PostgreSQL is the primary database").
Local dev DB	PGlite (embedded WASM Postgres)	Zero-install developer experience — `npm install && npm run dev` works with no local Postgres. Production requires a real `DATABASE_URL` (enforced at runtime).
Auth	Custom session-cookie auth, scrypt password hashing	EPS Ch.13 specifies a single shared employee account per firm — no off-the-shelf multi-user auth library fit that model.
Styling	Tailwind CSS v4, existing design tokens	Reused the landing page's design system (Ch.26) for visual consistency.
Mutations	React 19 Server Actions (progressive enhancement forms)	Idiomatic for Next.js 16; every mutation is a real HTTP-testable POST, which is how all E2E verification was done.

3.2 Module Map

Matches the EPS Ch.7 module table directly:

EPS Module	Implementation
Authentication	`src/lib/auth/` (session, password, rate limiter), `src/app/login/`
Client Management	`src/app/(app)/clients/`, `src/app/(app)/services/`
Collection Engine	`src/lib/collectionRequestStateMachine.ts`, `src/app/(app)/collections/`
Messaging	`src/lib/conversationOrchestration.ts`, `src/lib/scheduler.ts` (WhatsApp transport mocked)
Document Processing	`src/lib/ai/` (classification mocked, pipeline logic real)

Storage	`src/lib/storage/driveAdapter.ts` (Drive transport mocked)
Dashboard	`src/app/(app)/dashboard/`, `src/app/(app)/audit/`

3.3 Architectural Decisions Worth Knowing

- **Modular monolith** (AR-7.1) — one Next.js app, no microservices, matching pilot scope explicitly.
- **Mock-first integrations** — agreed with the user early: Google Drive, WhatsApp, and AI/OCR are all mocked behind adapter modules with the exact interface a real provider would need, because real credentials for any of them weren't available in this environment. See Section 9.
- **Single shared DB, org-scoped rows** — rather than one deployment per firm (which the EPS Ch.21 roadmap table literally shows for the pilot), every table carries `organization_id` and every query is scoped to it. This was an explicit decision the user confirmed, trading off against the "isolated deployment per firm" wording in favor of lower operational overhead for a pilot with few firms.
- **No public signup** — organizations are provisioned only via an internal CLI script (`npm run org:create`), never a public form. Also an explicit user decision, matching the EPS's sales-assisted tone rather than self-serve SaaS.
- **PGlite single-connection constraint** — discovered during testing: PGlite (used for local dev) does not reliably support two separate Node processes holding the same database directory open concurrently. All test mutations were routed through the running dev server's own HTTP endpoints rather than a parallel script once this was found.

4. Database & Data Model

13 tables across 7 migrations (`drizzle/0000` – `0006`), matching the EPS Ch.4/Ch.8 domain model plus the operational additions from later chapters (business-hours config, audit trail precision, Drive folder mapping).

Table	Purpose	Added
`organizations`	Tenant root. Also carries onboarding/integration status (Ch.3) and Ch.18 scheduling config (business hours/days, reminder interval, inactivity timeout).	M1, extended M4 & M8
`users`	The single shared employee account per organization (BR-13.1) — one row per org, not per person.	M2
`sessions`	DB-backed session tokens (cookie-based auth).	M2
`audit_logs`	Immutable event log (Ch.17). Insert-only. Carries `organization_id`, `client_id`, `collection_request_id` (added M11 for FR-17.3 precision), actor type/user, event type, description, optional metadata.	M2, extended M11
`clients`	Business records only, never authenticate (Ch.2). Unique `(organization_id, phone)` — one client per WhatsApp number. Carries `drive_folder_id` (M5).	M3, extended M4 & M5
`services`	Recurring document-requirement definitions (BR-001: "Services define requirements; clients do not").	M3
`service_document_requirements`	Per-service requirement templates (e.g. "Bank Statement").	M3
`client_services`	Many-to-many Client↔Service assignment.	M3
`collection_requests`	One collection cycle for one client/service/period. Ch.6 state machine status enum.	M3
`collection_request_requirements`	Per-cycle *snapshot* of the service's requirement templates (BR-002: historical requests are immutable to later service edits).	M3
`documents`	Metadata + Google Drive reference (never file bytes). Ch.6 status enum plus `deleted_from_drive` (Ch.14) with a `drive_deleted_at` timestamp.	M3, extended M5
`conversations`	One WhatsApp conversation per active Collection Request (BR-003). Ch.6 status enum including `human_control`.	M3
`messages`	Inbound/outbound message log, actor-typed sender.	M3

Every table's primary key is a UUID (DB-8.2). Every business table carries `organization_id` (DB-8.4). Foreign keys use `onDelete: "cascade"` for true parent/child ownership (e.g. a deleted org cascades everything) but deliberately `onDelete: "restrict"` (the default) for business-history references — a Client or Service with real Collection Request history cannot be hard-deleted, surfaced as a friendly UI error rather than a crash. `audit_logs` is the one exception: its client/user references use `onDelete: "set null"`, since a deleted client must never become permanently undeletable-in-spirit just because it was once mentioned in a log entry (a real bug found and fixed in M3 — see Section 5).

5. Milestone-by-Milestone Build Log

M1 — Foundations & Environment

EPS Ch.7, Ch.19

Postgres/Drizzle data layer with automatic dev/prod driver selection (`src/db/index.ts`): production requires a real `DATABASE_URL` and refuses to start without one; local dev falls back to embedded PGlite with zero setup. Env var scaffolding (`.env.example`), migration tooling (`npm run db:generate` / `db:migrate`), and the `(app)` route group established as the internal-app boundary separate from the untouched marketing site.

Bug found & fixed: Turbopack's dev bundling broke PGlite's WASM/filesystem asset loading with an unrelated-looking "path argument" TypeError on every DB call — fixed via `serverExternalPackages` in `next.config.ts` . `next build` 's static analysis never exercises this path (DB routes are `force-dynamic`), so it only surfaced under `next dev` .

M2 — Auth, Organization & Audit Log

EPS Ch.2, Ch.13, Ch.17

Shared-account login (scrypt password hashing, DB-backed sessions, `requireSession()` guarding every protected route), Organization + shared User creation (via the internal CLI provisioning script, per the no-public-signup decision), and `recordAuditEvent()` as the single insert-only write path onto the audit log.

Verified end-to-end over real HTTP: login reject/accept, protected-route redirect, logout, re-redirect after logout — including correctly decoding Next.js Server Actions' multipart wire format for scripted testing (not a trivial form POST).

M3 — Core Domain Schema + Client/Service CRUD

EPS Ch.4, Ch.8

Full schema for every remaining entity. Hand-managed CRUD only for Clients and Services (the two entities an admin actually creates directly) — Collection Requests/Documents/Conversations/Messages get their tables now but no "create" UI, since they're generated by the automation engine in later milestones, not typed in by hand.

Bug found & fixed: `audit_logs.client_id` had no `onDelete` behavior, so logging a client's own creation event (which happens immediately) made that client permanently undeletable forever after — the FK blocked it. Fixed with `onDelete: "set null"` : the log entry and its description text survive, only the structured reference clears.

M4 — Onboarding

EPS Ch.3

Mocked Google/WhatsApp connection toggles, CSV client import (hand-written parser — see the Excel note in Section 11), onboarding summary, and the BR-001 automation-activation gate enforced server-side (not just by disabling a button). `clients.phone` made unique per organization, with friendly duplicate-handling in both the CSV import and the manual client form.

M5 — Google Drive Storage (mocked)

EPS Ch.14, PR-005

`src/lib/storage/driveAdapter.ts` : `ensureClientFolder` (BR-3.003 — stores the folder ID, not its name, created lazily) and `uploadDocument`, wired to the real approval path (document approval in both the manual-add and review-document flows triggers upload the moment status becomes `approved`, per BR-11.5). Ch.14's manual-deletion reconciliation is fully implemented: a document "deleted from Drive" keeps its database row, flips status, and the UI shows "Deleted manually on DD/MM/YYYY HH:MM."

M6 — Scheduler (the real automatic trigger)

EPS Ch.5, Ch.16, Ch.18

`src/lib/scheduler.ts` + `POST /api/cron/tick` (bearer-token gated via `CRON_SECRET`, required in production): finds conversations idle past the org's configured inactivity timeout and evaluates them; finds conversations stuck waiting for a client reply past the reminder interval and sends a reminder. This is the actual automatic trigger that M8's manual "run now" buttons had stood in for.

Bug found & fixed: the evaluation function updated the Conversation's status but never the Collection Request's own status (also part of the Ch.6 state machine) — meaning the scheduler's reminder query, which correctly checks the Collection Request status, could never have matched anything. Both now transition together.

M7 — Collection Request Engine & State Machine

EPS Ch.5, Ch.6

Reordered ahead of M5/M6 mid-build (user-approved) once it became clear neither Drive nor WhatsApp integration had a real caller yet. The FR-6.2 transition graph (`draft → active → waiting_for_client → processing → completed / escalated / cancelled` , with completion reachable only via `processing` , and reopening the one deliberate exception back to `active`), plus the completion gate (BR-6.2: processing documents block completion; BR-11.2/BR-6.1: every requirement needs an approved document). Manual document marking (BR-11.3) lets an employee attach a document with a status directly — a real, spec-anticipated capability independent of the AI pipeline, not a placeholder for it.

M8 — Conversation Orchestration

EPS Ch.10, Ch.16, Ch.18

`src/lib/conversationOrchestration.ts` is the WhatsApp transport boundary — one function (`sendOutboundMessage`) is the single call site a real Business API integration replaces. Real logic: `ensureConversation` (BR-003), the initial-request send, identical-filename duplicate detection (later superseded by M9's fuzzier version), `evaluateAndPrompt` (the "after N minutes of inactivity" decision, silent when unsatisfied), and `reopenIfCompleted` (a new document after completion reopens the request automatically). Since there was no scheduler yet at this point, the inactivity timer and inbound WhatsApp webhook were both stood in for by explicit buttons on the Collection Request page — M6 later supplied the real automatic trigger for the same functions.

M9 — AI Document Intelligence (mocked)

EPS Ch.9, Ch.11

`src/lib/ai/intentClassifier.ts` and `documentClassifier.ts`: no OCR/LLM provider is configured, so both use deterministic rules (keyword matching for intent; filename-token overlap against candidate requirement names for document matching) behind the exact interface a real provider would implement. The full Ch.11 pipeline is wired for real: unsupported file types and unreadable files are rejected/flagged automatically before ever creating a document row; a confident match (≥60%) auto-approves and uploads to Drive; anything less certain — or unmatched entirely — is still filed but lands in `needs_review` for a person to resolve.

Gap found & fixed: a document classification couldn't confidently match had nowhere to render at all — the page only ever showed documents grouped under a requirement. Added an "Unmatched documents" section with a manual assign-to-requirement action.

M10 — Operational Dashboard

EPS Ch.12

Five status cards (Active, Waiting for Client, Needs Employee Review, Documents in Processing, Completed Today) each with count and percentage of all active collections (FR-12.2). "Needs Employee Review" is a proper union (not a naive sum) of escalated requests and requests with a `needs_review` document, so a request matching both criteria only counts once. Drill-down per card (FR-12.3) shows client, progress %, missing documents, last activity, status — not a full client list. Client search by name or phone (FR-12.5).

M11 — Audit Trail UI + Resilience

EPS Ch.15, Ch.17

The audit log itself existed since M2 but nothing ever displayed it — closed with an org-wide `/audit` feed and a per-Collection-Request "Audit History" section (FR-17.3), which required adding `collection_request_id` to the schema since `client_id` alone wasn't precise enough (one client can have many requests over time). `src/lib/resilience.ts` (`withRetry`): exponential-backoff retry wrapper wired into the Drive upload call site; callers catch failures and degrade gracefully (log + leave the document approved-but-unfiled) rather than crashing the action or corrupting the Collection Request (BR-15.1).

M12 — Security Hardening

EPS Ch.13, NFR-24.4

See Section 8 for full detail. Found and fixed a severe cross-tenant IDOR affecting document approval and client/service CRUD; fixed a related authorization gap in the scheduler's manual trigger; added login rate limiting (5 attempts / 15 minutes) and standard security response headers.

M13 — Full Pilot Acceptance Validation

EPS Ch.20

One continuous 32-assertion scripted run through the entire pilot lifecycle in a single organization — onboarding through business-hours config, service/requirement setup, CSV client import, collection request creation, conversation initiation, AI document classification (both auto-approved and needs-review paths), dashboard queue visibility, scheduler-driven prompting, client-confirmed completion, Drive deletion reconciliation, request reopening, and full audit trail verification. All 32 checks passed. Confirmed via `git diff` against the initial commit that the marketing landing page has zero changes across the entire build.

6. Routes, Pages & Server Actions Reference

6.1 Pages

Route	Purpose	Auth
`/`	Marketing landing page (pre-existing, untouched)	Public
`/login`	Shared-account login	Public (redirects away if already authenticated)
`/dashboard`	Work-queue status cards, drill-down, client search	Protected
`/onboarding`	Integration toggles, CSV import, automation activation	Protected
`/clients`, `/clients/new`, `/clients/[id]`	Client CRUD, service assignment, collection-request creation	Protected
`/services`, `/services/new`, `/services/[id]`	Service CRUD, requirement templates	Protected
`/collections`, `/collections/[id]`	Collection Request list/detail — status transitions, requirements, documents, conversation timeline, audit history	Protected
`/audit`	Org-wide audit log	Protected
`/settings`	Business-hours/scheduling config, manual scheduler trigger	Protected

6.2 API Routes

Route	Method	Purpose
`/api/health`	GET	DB connectivity check (ops monitoring)
`/api/cron/tick`	POST	External-scheduler-callable endpoint running the real automatic evaluation/reminder trigger. Bearer-token gated by `CRON_SECRET`.

6.3 Server Action Files

File	Actions
`src/app/login/actions.ts`	`login` (rate-limited)
`src/app/(app)/actions.ts`	`logout`

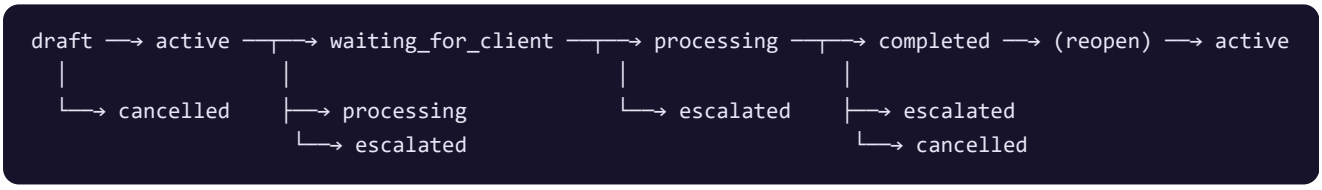
`src/app/(app)/clients/actions.ts`	`createClient`, `updateClient`, `deleteClient`, `assignService`, `unassignService`
`src/app/(app)/services/actions.ts`	`createService`, `updateService`, `deleteService`, `addRequirement`, `removeRequirement`
`src/app/(app)/collections/actions.ts`	`createCollectionRequest`, `transitionStatus`, `addManualDocument`, `reviewDocument`, `assignDocumentRequirement`, `simulateDriveDeletion`
`src/app/(app)/collections/conversationActions.ts`	`initiateConversation`, `simulateInboundMessage`, `evaluateNow`, `markFinished`, `markMoreDocuments`, `takeOverConversation`, `releaseConversation`, `sendEmployeeMessage`
`src/app/(app)/onboarding/actions.ts`	`connectGoogle` / `disconnectGoogle`, `connectWhatsapp` / `disconnectWhatsapp`, `activateAutomation`, `deactivateAutomation`, `importClients`
`src/app/(app)/settings/actions.ts`	`updateBusinessHours`, `runSchedulerNow`

7. Core Workflows

7.1 Document Collection Lifecycle (Ch.5/Ch.6/Ch.10)

1. Employee assigns a Service to a Client, then starts a Collection Request for a period (or, once a scheduler-driven auto-create trigger is wired in a future milestone, this happens on its own).
2. The Service's requirement templates are snapshotted into the request (BR-002) — later edits to the Service template never retroactively change this request.
3. Employee (or, once real WhatsApp is connected, the automated flow) initiates the conversation — the first outbound message.
4. Documents arrive (simulated inbound messages today); each runs through the Ch.11 pipeline — validated, classified, matched, auto-approved or flagged for review, stored to (mock) Drive.
5. Once all requirements are satisfied, the scheduler (or an idle-triggered evaluation) prompts the client to confirm Finished / More Documents — never assumed automatically (BR-16.1).
6. A Finished reply walks the request through `processing → completed`; a new document after completion reopens it automatically.

7.2 Collection Request State Machine



8. Security Review & the Cross-Tenant Fix

CRITICAL — FOUND AND FIXED

As part of M12, a dedicated audit was run specifically to check EPS NFR-24.4 ("each accounting firm shall operate in a fully isolated environment"): every database read/write reachable from a route param or bound server-action argument was checked for whether it verified the resource belonged to the caller's `organization_id`.

What was found

Most mutations that took a resource ID as a bound server-action argument fetched or mutated the row by ID *alone*, without checking organization ownership:

- `clients/actions.ts`: `updateClient`, `deleteClient`, `assignService`, `unassignService`
- `services/actions.ts`: `updateService`, `deleteService`, `addRequirement`, `removeRequirement`
- `collections/actions.ts`: `reviewDocument`, `assignDocumentRequirement`, `simulateDriveDeletion`, `addManualDocument`

The concrete exploit: an authenticated employee of Organization A could view, edit, approve, reject, or delete Organization B's clients, services, and documents by substituting a foreign ID into an otherwise-legitimate request — the most severe form being direct cross-tenant approval/rejection of another firm's client documents.

A related, separate issue: the `/settings` "run scheduler now" button called the exact same function the global cron endpoint uses, so any employee's click would trigger message sends and evaluations across *every* organization in the database, not just their own.

What was fixed

Every affected query now scopes explicitly to `organization_id = session.organizationId`, either directly or via shared helpers (`getOrgScopedCollectionRequest`, `getScopedDocument`, `getOrgScopedService`) that also verify parent-child ownership — e.g. a document must belong to the *specific* collection request it's addressed through, not just exist somewhere in the org. `createCollectionRequest` additionally verifies both the client and service belong to the caller's org before trusting a `client_services` assignment row. `runScheduledTasks` now takes an optional `organizationId` to scope a single run; only the cron endpoint omits it.

How it was verified

1. Two real, separate organizations were provisioned. Direct-URL access from Org B to Org A's client/collection-request/service by ID correctly returned 404 in every case.
2. A genuine wire-level forgery: Org B's own legitimately-issued server-action reference (a real, validly-signed action ID) was submitted with the UUID *inside* the bound-argument payload swapped for Org A's client ID — not just a different URL, which Next.js Server Actions don't route by. The forged delete request returned normally (200/303) but affected zero rows; Org A's client was confirmed to still exist immediately after.

3. A full regression pass of every normal same-org CRUD/assignment/collection-request/document-approval flow (15 assertions) confirmed the added checks don't break legitimate access.

Additional hardening in the same milestone

Control	Detail
Login rate limiting	5 failed attempts per 15-minute window locks out that email, even against a subsequently-correct password. Process-local (acceptable for a single-instance pilot per BR-13.2).
Security headers	`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy`, `Strict-Transport-Security` — set on every response via `next.config.ts`.
Secrets audit	Confirmed no hardcoded credentials anywhere in source; only `.env.example` is tracked in git.
Password storage	Node's native `scrypt` (salted, timing-safe comparison) — no external dependency needed.

9. Integrations — Mocked vs. Real

Integration	Status	Real interface implemented?	What's needed for real
Google Drive	MOCKED	Yes — `ensureClientFolder` / `uploadDocument` in `driveAdapter.ts`	Google Cloud project, OAuth client credentials, Drive API scope grant
WhatsApp Business API	MOCKED	Yes — `sendOutboundMessage` in `conversationOrchestration.ts` is the single call site	Meta Business verification (has its own real-world lead time), WhatsApp Business API access, webhook endpoint for inbound messages
AI/OCR document classification	MOCKED	Yes — `classifyDocument` / `classifyIntent` in `src/lib/ai/`	An LLM API key (e.g. Anthropic) plus an OCR provider for image-based documents
Authentication	REAL	—	Nothing — fully implemented, not a stand-in
Audit logging	REAL	—	Nothing
Scheduler / cron trigger	REAL	—	An actual external cron caller (Vercel Cron, GitHub Actions scheduled workflow, etc.) needs to be pointed at `POST /api/cron/tick` — the endpoint itself is production-ready
State machines, business rules, resilience/retry	REAL	—	Nothing

> The mocking pattern is consistent throughout: each mock lives behind a small adapter module exposing the same function signatures a real integration would need, and every caller only ever touches the returned ID/result — never anything mock-specific. Swapping in a real provider is a change contained to that one file per integration.

10. Testing Performed

Every milestone was verified with a scripted, real end-to-end HTTP test — not just `npm run build` passing. Each test logs into a live dev server, drives the actual UI/forms via genuine POST requests (including correctly decoding Next.js Server Actions' multipart bound-argument wire format), and asserts on the resulting page content and/or direct database state. No test was declared passing without being run and observed green.

Milestone	What was tested	Assertions
M1	DB connectivity, migration tooling, build correctness	Direct connection + query round-trip
M2	Login/logout, session cookie behavior, protected-route redirects, audit trail accuracy	Full HTTP flow, cookie inspection
M3	Client/Service CRUD, assignment, deletion blocking, audit FK fix	11 checks
M4	Onboarding gate, CSV import (valid/invalid/duplicate rows), duplicate-phone rejection	15 checks
M5	Drive upload on approval, Drive-link display, deletion reconciliation	8 checks
M6	Scheduler evaluation branch, reminder branch, CRON_SECRET auth (401/200)	9 checks + auth verification
M7	Full state machine walk, invalid-transition blocking, completion gate, reopen	18 checks
M8	Conversation flow end-to-end, duplicate detection, human takeover/release	25 checks
M9	Unsupported/unreadable rejection, auto-approval, fuzzy duplicates, unmatched-document handling	13 checks
M10	Dashboard queue accuracy, search, queue transitions on status change	10 checks
M11	Org-wide and per-request audit history, cross-request isolation, resilience utility (4 unit-style checks)	12 checks
M12	Cross-org attack simulation (2 real orgs, forged bound arguments), rate limiting, security headers, full same-org regression	~20 checks
M13	Single continuous full-lifecycle run spanning every milestone's feature in one organization	32 checks

Total: 190+ individual scripted assertions across all milestones, all passing at the time of writing.

> **Test harness note:** several genuine test-script bugs were found and fixed along the way (ambiguous form-marker matching when a page has multiple similar forms; React's SSR comment-marker text interpolation breaking naive substring checks; forgetting to set a required form field). Each is called out in the relevant milestone's commit message where it could be mistaken for an app bug. Two real *application* bugs were also

found this way and are documented in Section 5 (the audit-log FK issue in M3, the scheduler status-sync issue in M6) alongside the Section 8 security finding.

11. Known Limitations & Deferred Scope

Item	Why
Excel (.xlsx) import not implemented — CSV only	The only npm-available parser (`xlsx` /SheetJS) carries unpatched high-severity vulnerabilities (prototype pollution, ReDoS) since SheetJS moved patched releases off the public registry. CSV covers the same functional need (FR-005) without that risk; revisit if a trustworthy parser becomes available.
Google Drive, WhatsApp, AI/OCR are mocked	No credentials available in this environment. See Section 9 for exactly what's needed to make each real.
Rate limiting is process-local (in-memory)	Sufficient for a single-instance pilot (BR-13.2); would need a shared store (Redis, etc.) if ever deployed as multiple instances.
No automatic scheduled Collection Request creation	Ch.5's "automatic" trigger for *starting* a new cycle each period wasn't built — Collection Requests are created manually (or would need a small addition to the scheduler once business-hours/frequency config per-service is defined). Everything *after* creation (conversation, documents, completion, reminders) is fully automatic.
"Documents in Processing" dashboard card typically reads zero	The mocked AI classification resolves synchronously, so there's never a persisted "processing" window to observe. Wired correctly; will show real data once/if an async OCR job queue exists.
Production deployment not executed	No hosting credentials provided; the app is deployment-ready (env var separation, health check, migration scripts, CRON_SECRET-gated endpoint) but was not pushed to a live host as part of this build.

12. Files Created / Changed

74 files changed since the initial landing-page commit (+19,617 / −4,053 lines). Organized by area:

Database

`src/db/schema.ts`, `src/db/index.ts`, `drizzle/0000` – `0006` (7 migrations + snapshots), `drizzle.config.ts`, `scripts/migrate.ts`, `scripts/create-organization.ts`

Auth & Security

`src/lib/auth/session.ts`, `password.ts`, `rateLimiter.ts`, `src/app/login/` (page, form, actions)

Domain Logic

`src/lib/collectionRequestStateMachine.ts`, `conversationOrchestration.ts`, `scheduler.ts`, `businessHours.ts`, `resilience.ts`, `audit.ts`, `csv.ts`

AI (mocked)

`src/lib/ai/intentClassifier.ts`, `documentClassifier.ts`

Storage (mocked)

`src/lib/storage/driveAdapter.ts`

Data Access Layer

`src/lib/data/clients.ts`, `services.ts`, `collectionRequests.ts`, `dashboard.ts`, `auditLog.ts`, `organizations.ts`

App Routes & UI

`src/app/(app)/` — `layout.tsx`, `actions.ts`, and full subtrees for `dashboard/`, `onboarding/`, `clients/`, `services/`, `collections/`, `audit/`, `settings/`; `src/app/api/health/`, `src/app/api/cron/tick/`

Config

`next.config.ts`, `.env.example`, `package.json` (scripts)

Untouched (verified via git diff)

`src/app/page.tsx`, `src/app/layout.tsx`, all of `src/components/landing/`

13. Recommended Next Steps

1. **Decide on real credentials** for Google Drive, WhatsApp Business API, and an AI/OCR provider — each swap is contained to one adapter file (Section 9), but Meta's Business verification in particular has its own lead time worth starting early.
2. **Choose a hosting target** and provision a real managed Postgres instance; the app already enforces this split (refuses to start in production without `DATABASE_URL`) and ships a health-check endpoint for platform monitoring.
3. **Wire the external cron** (Vercel Cron, GitHub Actions scheduled workflow, or equivalent) to `POST /api/cron/tick` with the `CRON_SECRET` bearer token.
4. **Consider Excel import** once a trustworthy, actively-maintained parser is available, or if CSV-only proves insufficient for pilot firms in practice.
5. **Run the pilot** against Ch.20's success metrics (SM-20.1–20.4) with a real accounting firm once the above integrations are live — the dashboard, audit trail, and state machine are all in place to observe those metrics directly.